



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

Département de génie informatique et génie logiciel

Cours INF1900:
Projet initial de système embarqué

Travaux pratiques 7 et 8

Production de librairie statique et stratégie de débogage

Par l'équipe

No. 107111

Noms:

Anis Benabdallah
Jérémie Anglaret-Guirguis
Marc Abou-Saada
Yanis Ben Boudaoud

Date:

16 mars 2026

Classe **Timer**

La classe `Timer` sert à configurer les trois minuteries matérielles de l'ATmega, soit `Timer0`, `Timer1` et `Timer2`, en mode PWM ou en mode CTC. Comme ces timers se configurent de façon assez semblable, ils ont été regroupés dans une même classe. Le choix du timer se fait avec l'enum `Id`, ce qui évite d'avoir une classe différente pour chaque timer.

Registres affectés :

La classe modifie les registres `TCCRnA` et `TCCRnB` pour choisir le mode de fonctionnement et le prescaler. Elle écrit aussi dans `OCRnA` et `OCRnB` pour les valeurs de comparaison. Lorsqu'on appelle `setOCRA()` ou `setOCRB()`, le compteur `TCNTn` est remis à zéro avant l'écriture. En mode CTC, la classe active aussi les interruptions de comparaison dans `TIMSK0`, `TIMSK1` ou `TIMSK2`. Les bits touchés sont donc surtout ceux du mode WGM, du prescaler CS, des sorties de comparaison COM en mode PWM et des interruptions de comparaison.

Broches I/O :

En mode PWM, les broches de sortie du timer doivent être configurées en sortie par l'utilisateur avant l'utilisation. Pour `Timer0`, il s'agit de `OC0A` (PB3) et `OC0B` (PB4). Pour `Timer1`, ce sont `OC1A` (PD5) et `OC1B` (PD4). Pour `Timer2`, ce sont `OC2A` (PD6) et `OC2B` (PD7). En mode CTC, aucune broche particulière n'est nécessaire.

Méthodes Notables :

- Les méthodes `setModePWM()`, `setModeCTC()`, `setOCRA()`, `setOCRB()`, `startTimer()` et `stopTimer()` sont non bloquantes. Elles font seulement des écritures dans les registres et ne contiennent aucun délai.
- `setModeCTC()` active les interruptions de comparaison A et B du timer choisi. Si ce mode est utilisé avec interruptions, il faut donc déclarer les ISR correspondantes et activer les interruptions globales avec `sei()` dans le code appelant. `startTimer()` est nécessaire pour démarrer réellement le timer, alors que `stopTimer()` enlève simplement les bits du prescaler sans effacer le reste de la configuration.

Enums :

`Id` permet de choisir entre `TIMER0`, `TIMER1` et `TIMER2`. `PWMMode` permet de choisir entre `PHASE_CORRECT` et `FAST`. `Prescaler` permet de choisir le facteur d'horloge : pas de prescaler, /8, /64, /256 ou /1024. Les valeurs de comparaison dépendent du timer utilisé : 0 à 255 pour `Timer0` et `Timer2`, puis 0 à 65535 pour `Timer1`.

Ordre d'utilisation :

En mode PWM, on configure d'abord le mode avec `setModePWM()`, on écrit ensuite la valeur voulue avec `setOCRA()` ou `setOCRB()`, puis on démarre le timer avec `startTimer()`. En mode CTC, on appelle `setModeCTC()`, on fixe la ou les valeurs de comparaison, on déclare les ISR né-

cessaires, on active les interruptions globales avec `sei()`, puis on démarre le timer avec `startTimer()`. Un appel à `stopTimer()` arrête le timer, mais la configuration reste en place pour un redémarrage plus tard.

Classe **Wheel**

La classe **Wheel** sert à contrôler une roue en gérant à la fois le sens de rotation et la vitesse. Le sens est contrôlé avec une broche de direction sur `PORTD`, tandis que la vitesse est réglée avec un objet **Timer** passé par référence au constructeur. Cette classe ne configure pas elle-même la minuterie en mode PWM, elle suppose qu'il a déjà été préparé ailleurs. Elle est utilisée par la classe **Motor**, qui lui fournit la minuterie et choisit les bonnes broches. La classe est donc réutilisable pour les deux roues, mais elle reste liée au branchement choisi sur `PORTD`.

Registres affectés :

La classe modifie directement `DDRD` et `PORTD`. Dans le constructeur, elle met en sortie la broche de direction ainsi que la broche PWM associée. Les méthodes `goForward()` et `goBackward()` modifient `PORTD` pour choisir le sens de rotation. La vitesse est changée indirectement par `adjustSpeedValue()`, qui appelle `setOCRA()` ou `setOCRB()` de la minuterie selon le mode choisi. Cela affecte donc aussi les registres de comparaison de la minuterie utilisée.

Broches I/O :

Cette classe utilise uniquement `PORTD`. Si la broche de direction choisie est `PD4`, la broche PWM associée est `PD6`. Si la broche de direction choisie est `PD5`, la broche PWM associée est `PD7`.

Aucune interaction avec CAN ou mémoire externe. USART : `PD0/PD1` sur `PORTD`, pas de conflit direct avec `PD4/PD5/PD6/PD7`.

Dépendances :

La classe dépend d'un objet **Timer** passé par référence au constructeur. Ce timer doit déjà être configuré en mode PWM et démarré avant l'utilisation, sinon le réglage de vitesse n'aura pas l'effet voulu. La broche de direction doit être `PD4` ou `PD5`, car ce sont les deux seuls cas prévus dans le code. Aucun ISR n'est requis.

Enums :

L'enum `OCR` permet de choisir entre le canal A et le canal B du timer. Ce choix détermine si la roue utilise `setOCRA()` ou `setOCRB()`. La variable `speedValue` est un `uint8_t`, donc son domaine va de 0 à 255. Une valeur de 0 correspond à l'arrêt, et une valeur plus élevée donne un rapport cyclique plus grand.

Classe **Motor**

La classe **Motor** sert à contrôler les deux roues du robot avec une seule interface. Elle contient deux objets **Wheel** et un objet **Timer** utilisé pour le PWM. Le timer est configuré directement dans le constructeur, ce qui rend l'objet utilisable dès sa création. `Timer2` est utilisé en mode `PHASE_CORRECT` avec un prescaler de 8, ce qui est suffisant pour le contrôle de vitesse des moteurs.

Registres affectés :

Ces registres sont affectés indirectement via les classes `Timer` et `Wheel`, : `TCCR2A`, `TCCR2B`, `OCR2A`, `OCR2B`, `TCNT2`, `DDRD`, `PORTD`

Broches I/O :

- La roue gauche utilise `PD5` pour la direction et `PD7` pour le PWM.
- La roue droite utilise `PD4` pour la direction et `PD6` pour le PWM.
- Ces quatre broches sont donc réservées au contrôle des moteurs. Aucune interaction avec CAN, mémoire externe ou USART.

Méthodes notables :

- Le constructeur configure automatiquement `Timer2` en mode PWM `PHASE_CORRECT`, avec un prescaler de 8, puis le démarre.
- `goForward()`, `goBackward()` et `stop()` : Méthodes non bloquantes et leur temps d'exécution est très court, puisqu'elles font seulement appel aux deux objets `Wheel`.
- Aucun ISR n'est requis. Les paramètres `speedValue1` et `speedValue2` sont de type `uint8_t`, donc leur domaine va de 0 à 255.

Dépendances :

La classe dépend de `Wheel` et de `Timer`, mais tout est pris en charge dans le constructeur. Aucune initialisation externe, aucun appel à `sei()` et aucune déclaration d'ISR ne sont nécessaires.

Classe LED

La classe LED sert à contrôler une LED bicolore reliée à deux broches d'un même port. Le constructeur reçoit le registre `PORTx` à utiliser ainsi que les deux broches, puis détermine automatiquement le registre `DDRx` correspondant avant de configurer ces broches en sortie. Cela permet d'utiliser la classe avec différents ports sans devoir la modifier. Les méthodes `off()`, `red()` et `green()` sont non bloquantes et leur temps d'exécution est très court. Aucun ISR n'est nécessaire.

Registres affectés :

La classe modifie les registres `PORTx` et `DDRx`. Les bits associés à `pinIn_` et `pinOut_` sont configurés en sortie dans le constructeur, puis modifiés dans `PORTx` pour choisir la couleur affichée ou éteindre la LED. Le registre `DDRx` utilisé est choisi automatiquement à partir du `PORTx` passé au constructeur. Par exemple, si le port donné est `PORTA`, alors le registre utilisé sera `DDRA`.

Broches I/O :

La classe utilise deux broches d'un même port, choisies lors de la construction de l'objet. Ces broches doivent être réservées à la LED et ne pas être partagées avec un autre périphérique pendant l'utilisation. Si la configuration actuelle utilise `PORTA`, `PA0` et `PA1`, ces deux broches sont alors réservées à la LED. Aucune interaction avec le CAN, la mémoire externe ou l'USART.

Méthodes notables :

amber() : La méthode est bloquante. Elle alterne entre **red()** et **green()** avec deux délais de 3 ms, pour un total d'environ 6 ms par appel. Pour obtenir un effet ambre visible, elle doit donc être appelée plusieurs fois de suite, généralement dans une boucle. Pendant l'exécution des délais, le programme reste occupé dans cette méthode. Les autres méthodes ne contiennent aucun délai.

Classe Bouton

La classe **Button** permet la gestion d'un bouton physique avec détection par interruption externe. Le support de trois emplacements (**MOTHERBOARD/PD2**, **PD3**, **PB2**) est géré via l'enum **Location**, ce qui permet de réutiliser la même interface indépendamment du câblage. Les valeurs de l'enum **Mode** encodent directement les bits **ISCnx**, évitant de faire ça manuellement. La configuration des interruptions est faite dans **init()** au lieu du constructeur, pour donner à l'utilisateur le contrôle du moment d'activation.

Registres affectés :

- **EICRA** : bits **ISC01** et **ISC00** pour **INT0**, **ISC11** et **ISC10** pour **INT1**, **ISC21** et **ISC20** pour **INT2** (contrôle du front de détection).
- **EIMSK** : bits **INT0**, **INT1** ou **INT2** selon l'emplacement
- **DDRx** : le bit correspondant au bouton est mis en entrée. **PINx** : lecture de l'état dans **isPressed()**. Le registre **SREG** est aussi affecté par l'appel à **sei()** dans **init()**.

Broches I/O :

- **MOTHERBOARD** utilise **PD2** (**INT0**) en entrée
- **PD3_BUTTON** utilise **PD3** (**INT1**), en entrée
- **PB2_BUTTON** utilise **PB2** (**INT2**), en entrée

Méthodes notables :

isPressed() : la méthode est bloquante, elle contient un **_delay_ms(30)** pour l'anti-rebond, donc chaque appel prend au minimum 30 ms. La logique de retour est inversée selon l'emplacement : active-high pour **MOTHERBOARD**, active-low pour **PD3** et **PB2**. À cause du délai de 30 ms, il faut éviter d'appeler cette méthode dans un ISR.

init() : non bloquante. Configure le pin en entrée (**DDR**), règle le mode de détection dans **EICRA** et active le masque dans **EIMSK**. Appelle **sei()**, ce qui active les interruptions globales pour tout le système. L'utilisation de **init()** nécessite d'avoir déclaré le bon ISR avant.

Enum :

L'enum **Location** permet de choisir la broche utilisée et l'interruption externe associée. L'enum **Mode** permet de choisir le front de détection : **RISING**, **FALLING** ou **ANY**.

Partie 2: Décrire les modifications apportées au Makefile de départ

Makefile lib :

Des modifications ont été apporté pour l'adapter à la création d'une librairie statique. Ces modifications simplifient la compilation et retirent les fonctionnalités inutiles pour une librairie statique.

Modifications apportées :

- Utilisation de la fonction wildcard pour récupérer les fichier sources automatiquement, pour éviter un ajout manuel des fichiers :

```
PRJSRC= $(wildcard *.cpp)
```

- Ajout de la variable qui créer des archives statiques (.a). Elle regroupe les fichiers objets (.o) en une bibliothèque (.a) :

```
AR=avr-ar
```

- La cible « **TRG=\$(PROJECTNAME).elf** » a été modifié pour générer une librairie statique au lieu d'un exécutable :

```
TRG=$(PROJECTNAME).a
```

- La commande « **\$(CC) \$(LDFLAGS) -o \$(TRG) \$(OBJDEPS)** » a été modifié pour archiver les fichiers objets dans la librairie statique

```
$(AR) -crs $(TRG) $(OBJDEPS)
```

Suppression de variable inutile :

- Les lignes si dessous ont été retiré puisque la librairie ne génère pas de fichiers hex:

Retirement de la variable « **\$(HEXROMTRG)** » de la cible « **all: \$(TRG) \$(HEXROMTRG)** ».

```
HEXROMTRG=$(PROJECTNAME).hex
```

```
HEXTRG=$(HEXROMTRG) $(PROJECTNAME).ee.hex
```

```
%.hex: %.elf
```

```
$(OBJCOPY) -j .text -j .data -O $(HEXFORMAT) $< $@
```

- Retrait de la commande « **install** »

Makefile exec :

Des modifications ont été apporté pour permettre la compilation du programme principale en utilisant ne bibliothèque statique externe.

Modifications apportées :

- Inclusion du dossier contenant les fichier d'entête de la bibliothèque, et permet de lier la bibliothèque (libstatique.a):

```
INC=-I ../lib/
```

```
LIBS=-L ../lib/ -lstatique
```

- Ajout de la cible pour permettre de compiler automatiquement la bibliothèque statique:

```
lib:
```

```
(cd ../lib/ && make)
```

- Modification de la cible all qui garantit la compilation de la bibliothèque avant le programme:

```
all: lib
all: $(TRG) $(HEXROMTRG)
```

- Modification de la règle de génération du fichier exécutable pour ajouter la dépendance a la bibliothèque:

```
$(TRG): lib $(OBJDEPS)
```

- Ajout de la cible debug pour activer le débogage lors de la compilation:

```
debug: CXXFLAGS += -DDEBUG -g
debug: CFLAGS += -DDEBUG -g
debug: clean
$(MAKE) -C ../lib/ CXXFLAGS="$(CXXFLAGS)" CFLAGS="$(CFLAGS)"
$(MAKE) install
serieViaUSB -l
```

- Modification de la cible clean pour nettoyer aussi les fichiers de la librairie:

```
$(MAKE) -C ../lib/ clean
```

Références

- Classe Memoire24cxx et classe can : Ces classes sont fournies pour notre utilisation.
- Microchip Corporation, 8-bit Atmel Microcontroller with 16/32/64/128K Bytes In System Programmable Flash – ATmega164A/PA/324A/PA/644A/PA/1284/P, Fiche technique, http://www1.microchip.com/downloads/en/devicedoc/atmel-8272-8-bit-avr-microcontrolleratmega164a_pa-324a_pa-644a_pa-1284_p_datasheet.pdf 659 p.